

Mizar Evo **の設計指針について**
自動証明・数学的記述・検証可能な計算の再接続

中正 和久

岩手県立大学

TPP 2026、**理化学研究所 AIP 東京オフィス**、2026 年 11 月 16–17 日

0.3 - 現行 Mizar の課題と 6 つの設計指針

現行 Mizar から取り組む課題	新仕様の設計指針
§ 1 基盤: 論理・MML の継承と処理系の刷新	一階論理・集合論を維持し、kernel を小さく保つ
§ 2 記述: 暗黙の型・登録・演算選択の追跡	数学的な抽象化を継承し、暗黙の選択を明示
§ 3 汎用化: 定義・定理・scheme の個別機構	template で共通化の仕組みを統一
§ 4 計算: 証明と実行可能な手続きの接続	algorithm の契約・不変条件・停止性を検査
§ 5 開発基盤: 依存管理・配布・差分検証・IDE	module・namespace・package・差分ビルド・LSP
§ 6 検査・自動化: 外部探索と検査根拠の接続	ATP を活用し、evidence のインスタンス化と SAT 検査

1.1 - 基盤の継承: 論理は保ち、処理系を再整備

数学的記述の層を継承し、
処理系と開発基盤を再構築する。

- 一階論理と Tarski-Grothendieck 集合論を基盤として維持。
- soft type、mode、attribute、registration、structure、**宣言的証明を継承。**
- **証明探索と検査を分離し、受理を小規模な kernel に集約。詳細は § 6。**

1.2 - 現行 Mizar: 集合論的な関数

関数適用は定義された集合論的關係 exact MML excerpt

```
func f.x -> set means  
:Def2:  
[x,it] in f if x in dom f otherwise it = {};
```

型付き関数は部分関数の上の soft type exact MML excerpt

```
definition  
  let X,Y;  
  mode Function of X,Y is quasi_total PartFunc of X,Y;  
end;
```

- 関数是对の集合として表される一階の対象。関数適用は集合論上で定義する。
- 関数の扱いに、基盤論理の高階化は不要。
- 記述上の課題: 定義域・グラフ・関数性・所属関係を直接記述すると煩雑。

2.1 - 記述の課題: 可読性と抽象化を継承・拡張

ソース上の関数表現 **sketch** (現行 Mizar と Mizar Evo で有効)

```
let X, Y be set;  
let f be Function of X, Y;  
let x be Element of X;  
... f.x ...
```

- 集合論的な関数を `Function of X,Y` と `f.x` で記述し、対や定義域の詳細を抽象化する。
- `soft type`、`mode`、`attribute`、`registration`、`scheme`、宣言的証明は、単なる記法の簡略化を超え、一階の集合論を可読な数学的記述へ結び付ける言語機構。
- 新仕様: 数学的な記述を継承・拡張し、型・登録・オーバーロードの暗黙の選択を明示・追跡可能にする。

2.2 - 暗黙の演算選択を明示・追跡する

環の二つの演算から、利用するビューを選択 sketch

```
let R be commutative Ring;  
f(R);                :: ambiguous Magma view  
f(R qua AddMagma);   :: use addition  
f(R qua MulMagma);   :: use multiplication
```

- 課題: 環は加法・乗法の両方で Magma を継承。f(R) だけではビューが定まらない。
- 新仕様: qua で加法・乗法のビューを明示し、曖昧な選択を解消。
- 自動適用した登録は適用経路を記録し、依存する規則を追跡可能にする。

3.1 - Template: functor の定義と結果型を共通化

例: 関数の和。「各点で値を足す」という構成を共通化。

汎用 functor **sketch** (存在・一意性の証明は省略)

```
definition
  let T be type extends non empty AddMagma;
  let I be non empty set;
  let f, g be Function of I, T;
  func AddDef: Add[T,I](f,g) -> Function of I,T means
    for i being Element of I holds it.i = T.add(f.i,g.i);
end;
```

- 現行 MML: 実数値・複素数値ごとに、和の結果型の再定義・登録が付随。
- 新仕様: 値の型 T をパラメータ化し、同じ構成の定義と結果型を共通化。

述語パラメータ付き定理としての帰納法 specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

- 述語パラメータにより、代入する述語ごとの一階定理の族を表現する。
- 明示的なインスタンス化: defpred に続けて by NatInduction[P], Base, Step。
- 関数子パラメータは集合ではなく schema レベルの記号として扱い、一階論理を維持する。
- 詳細: Białystok 資料 Story 6 (PermProduct[T]、qua によるビュー)、および本資料 Backup 4。

4.1 - 計算の課題: 検証可能な algorithm

- 現行 Mizar は宣言的証明と組込みの自動化を採用。ユーザ定義の tactic 言語は持たない。
- 課題: 数学的な証明と実行可能な計算を、一つの検査の枠組みに結び付ける。
- 新仕様: algorithm は契約を持つ手続き。正しさ・不変条件・停止性を検査する。
- 用途: アルゴリズム自体の検証と、検証済み手続きによる証明支援。

4.2 - Algorithm: 契約、証明、計算

契約付きのユークリッドの互除法 specification example

```
terminating algorithm euclid_gcd(a, b) -> Nat
  requires a >= 1 & b >= 1
  ensures result = Gcd(a, b)
do
  var x := a;  var y := b;
  while y <> 0 do
    invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);  decreasing y;
    const r := x mod y;  x := y;  y := r;
  end;
  return x;
end;
```

- 契約・不変条件・停止性の測度から一階の証明義務を生成し、定理と同じ枠組みで検査する。
- 二つの役割: 検証済みの自動化手続きと、by computation による検査済みの計算。仕様化済み。MVM 実行とコード抽出は今後の課題。

将来構想（現在の対応範囲には含まれない）：

- 整数論・組合せ論のアルゴリズム、記号計算、最適化手続き。
- 長期的な対象：暗号アルゴリズムとプロトコル、量子アルゴリズムと古典・量子ハイブリッド。

共通する検証・実行の流れ：手続き・契約の記述 → 不変条件・停止性の指定 → 証明義務の生成 → ATP による証明と kernel 検査 → 具体的な入力での実行 → コード抽出（将来）。

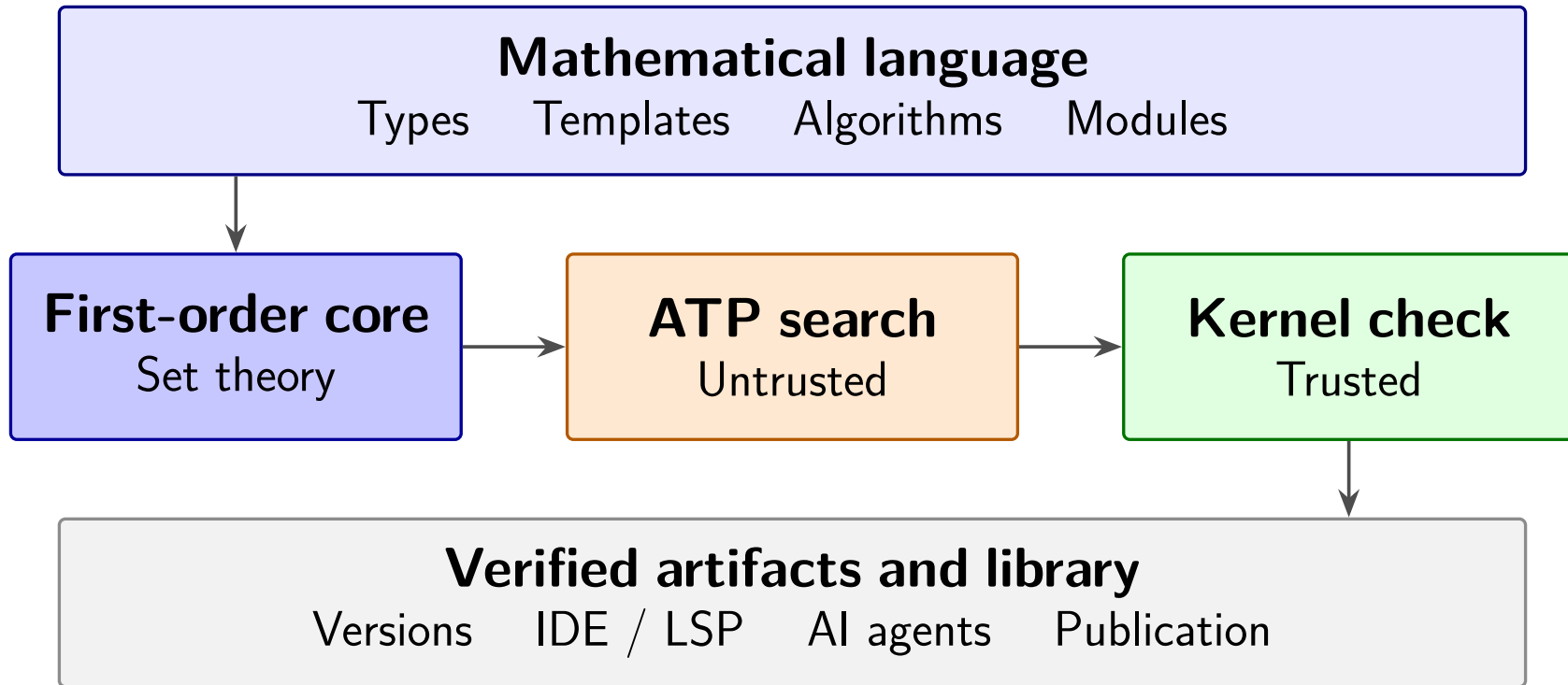
- **algorithm は、証明の自動化手続きと応用アルゴリズムの検証を共通の枠組みに結び付ける。対象の拡大は将来構想。**

5.1 - 開発基盤の課題と新仕様の対応

MML の 50 年の蓄積から見た課題	Mizar Evo
article が依存の単位 グローバルな名前管理 配布とバージョン管理が弱い フルビルド ATP は外付け、自動化が見えにくい IDE 連携と機械可読な入出力が弱い	明示的 import を持つモジュール namespace、完全修飾名 package、SemVer、lock file 依存指紋による差分ビルド 第一級の ATP パイプライン、解決トレース、 kernel evidence LSP、構造化診断、エージェント向けインタフェース

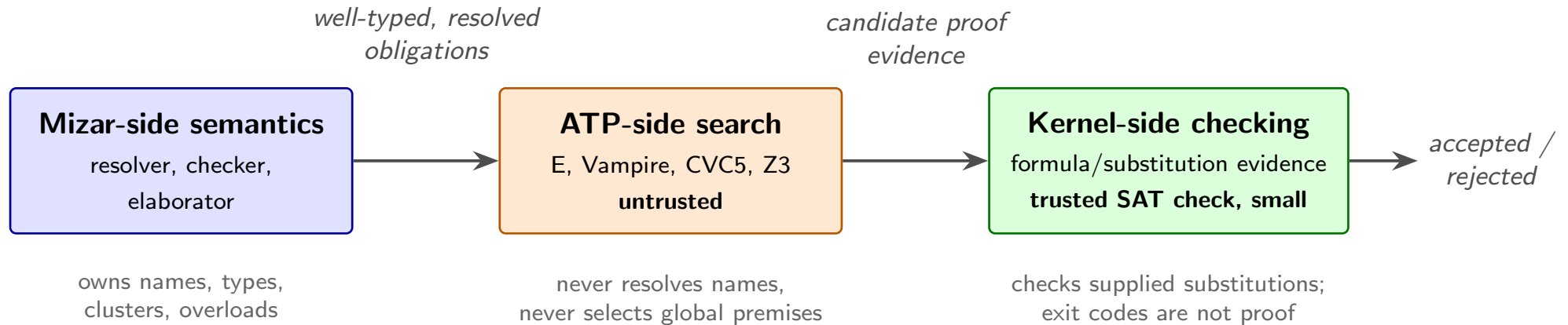
- Mizar の数学的思想を継承し、ソフトウェア構成は現代の開発要件に合わせて再設計する。
- 言語とともに開発基盤全体を現代化。各機能の詳細は Białystok 資料を参照。

5.2 - 全体像



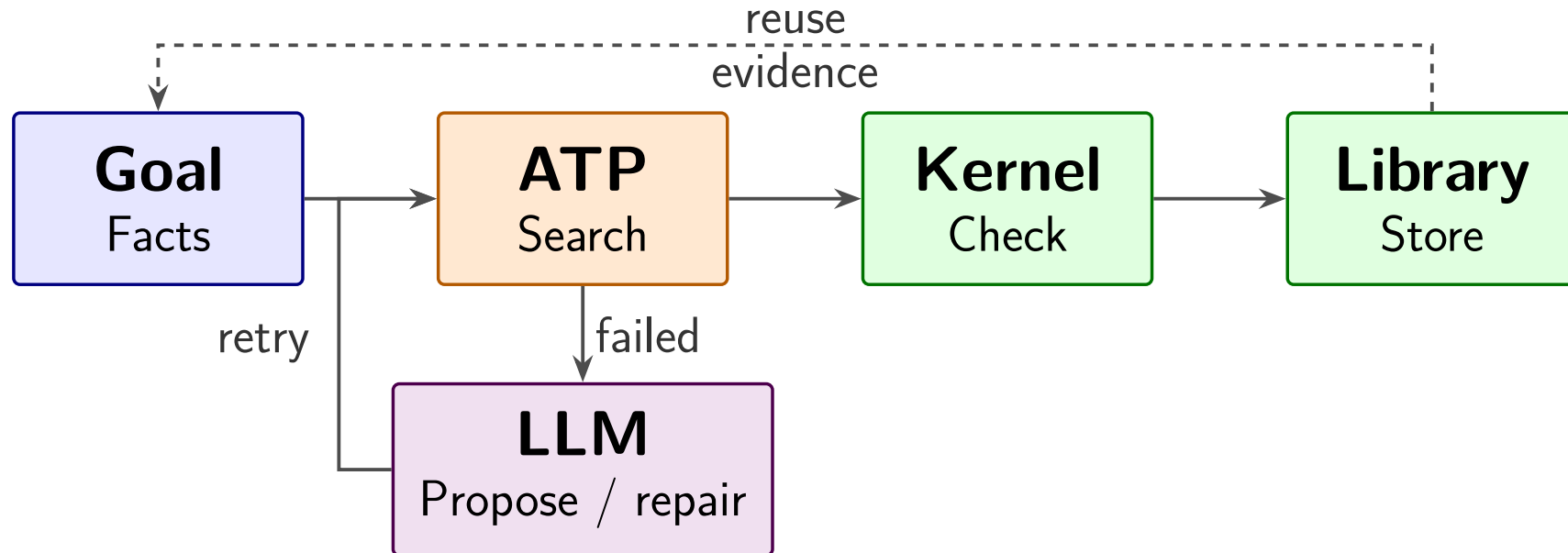
- 豊かな数学的記述を、小規模な一階論理の基盤と現代的な開発環境で支える。

6.1 - 自動化の課題: 証明探索と検査の分離



- 一階 ATP は証明探索を担う。探索結果は、信頼できる検査器による検証を要する。
- Mizar 側: 名前・型・cluster・オーバーロードの解決。ATP: 証明探索。kernel: 論理式と代入を、小規模な信頼できる SAT 検査で検証し、受理を判定。
- ATP の終了コードのみでは証明を受理しない。一階自動推論の導入によって信頼基盤を拡大しない設計。

6.2 - ATP の活用: 探索・検査・再利用



- ゴールと前提 → ATP 探索 → kernel 検査 → ライブラリへ保存・再利用。
- 未解決時は補題・方針を修正して再探索。図は設計意図。

6.3 - Evidence → インスタンス化 → SAT 検査

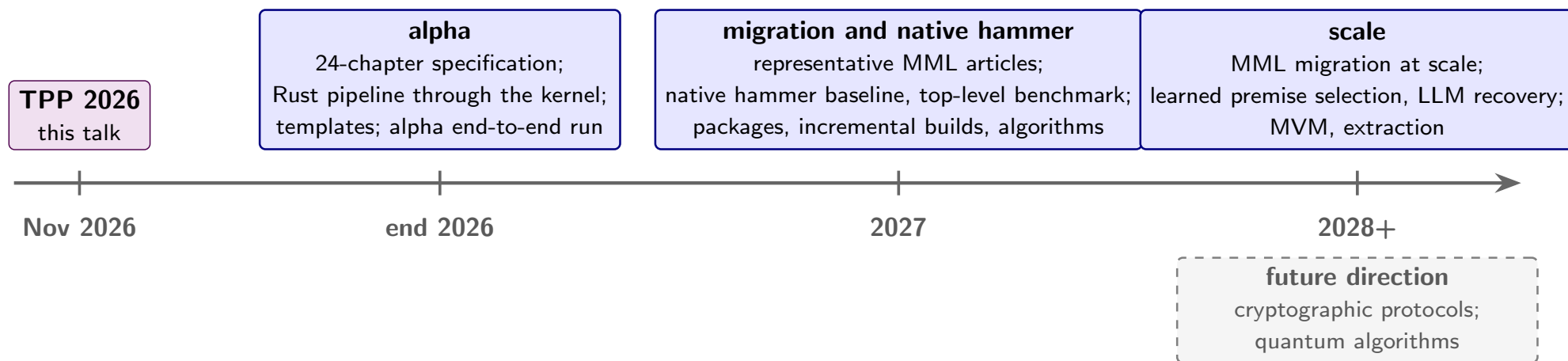
evidence → インスタンス化 → SAT 検査

1. **Evidence**: 元の論理式・明示的な代入・来歴・対象ゴールを保持。
2. **Kernel**: 対応関係と代入を検査し、論理式のインスタンスを生成。
3. **SAT**: インスタンスとゴールの否定を符号化。信頼できる検査器で UNSAT を確認した場合に受理。

仕様と実装の状況（2026 年 10 月）

- 仕様: 24 章と付録。英語が正典。
- main branch に実装済み: Rust フロントエンド（字句解析・構文解析・構文木）、alpha コーパス上の名前解決・型検査、証明義務の生成・決定的な discharge、ATP 問題の符号化・候補 evidence、SAT に基づく kernel の evidence 検査、キャッシュ・指紋・ビルドスケジューリングの各マイルストーン。
- 進行中: ソースから検証済み成果物までの end-to-end 統合、LSP サーバ、ドキュメント生成。
- 今後の課題: MVM 実行、コード抽出、ライブラリ全体の前提選択、MML 移行。
- 本講演の対象範囲に、外部 ATP を用いた end-to-end の実証結果は含めない。

ロードマップ



- 2026 年: 仕様、kernel までの Rust パイプライン、template 処理、alpha の end-to-end 実行の完成。
- 2027 年: 代表的な MML article の移行、native hammer のベースライン構築、MizAR と同じ top-level theorem 単位での評価。
- 2028 年以降: 移行の拡大、学習ベースの前提選択、LLM による失敗回復、MVM・コード抽出。暗号・量子は将来構想。

おわりに

*Keep the foundation small.
Keep the mathematics readable.
Modernize everything else.*

- 基盤を小さく保ち、数学的記述の可読性を維持し、周辺基盤を現代化する。
- 議論したい点: MML の移行、記述の利便性、自動化の評価。

Backup 1. MizAR 60 の詳細

Source: Jakubův et al., MizAR 60 for Mizar 50, ITP 2023.

- **データ**: MPTP で出力した MML 1147、**無名の top-level lemma を含む 57,897 件の定理**。MizAR 40 と同一版を用いた比較。
- **学習・開発・holdout を 90:5:5 に分割**。holdout 2,896 件中 1,690 件 (58.4%) を hammering mode で証明。**ユーザの前提指定なし、420 CPU 秒 (MizAR 40 は約 40.6%)**。
- **人または機械がライブラリから前提を選べる条件では 75% 超 (MizAR 40 は 56%)**。
- **最高性能の単一手法**: hammering mode で 30 秒 40%。**人間の前提指定ありで 120 秒 60%**。
- **転移**: 同手法を MML 1382 の新規 242 article、13,370 定理にも適用。
- **手法**: ENIGMA と Deepire で誘導した E と Vampire、学習ベースの前提選択、数百万の ATP 証明で学習するループ。

Backup 2. Sledgehammer 評価の詳細

評価	対象・方式	成功率
ITP 2022	AFP 5,000 ゴール、greedy 16 構成	68.8%
Magnushammer (2023 年公開)	PISA 1,000 定理、Sledgehammer	38.3%
同じ PISA 評価	学習による前提選択を用いる Magnushammer	59.5%

- ITP 2022: 50 entry × 100 ゴール。MePo、基準 512 事実、各構成 30 CPU 秒。構成は評価集合から事後選択。再構成は評価対象外。
- PISA: Isabelle2021-1、Sledgehammer の timeout 30 秒、ローカル 5 prover、複数設定の成功集合を集約。Isabelle 内での検証を成功条件とする。
- 年代が近くても、局所ゴールと定理全体、探索成功と検証済み証明を混同しない。

Backup 3. HOL から FOL への符号化と再構成

- 関数適用: 関数変数の適用を `app(F, X)` で表現。定数の適用はカーリー化を維持、または平坦化。
- ラムダ抽象: `lambda lifting` は定義式付きの新しい定数を導入する。コンビネータ変換が代替。
- 型: 多相な HOL の型を `guard`、`tag`、または単相化で符号化。選択は健全性・完全性・`prover` 性能に影響する。
- 真偽値: 論理式中の真偽値を返す項には追加の符号化が必要。
- 再構成: 使用した補題による `metis`、SMT 証明の `smt replay`、生成した `Isar` テキスト。再構成の失敗は別途集計。

Source: Meng and Paulson 2008; Blanchette, Böhme, Popescu, Smallbone 2016; Blanchette, Kaliszyk, Paulson, Urban 2016; Schurr, Fleury, Desharnais 2021.

Backup 4. Template のインスタンス化: 一つの証明、多くのビュー

有界な型パラメータと汎用的な定理 **specification example** (証明は省略)

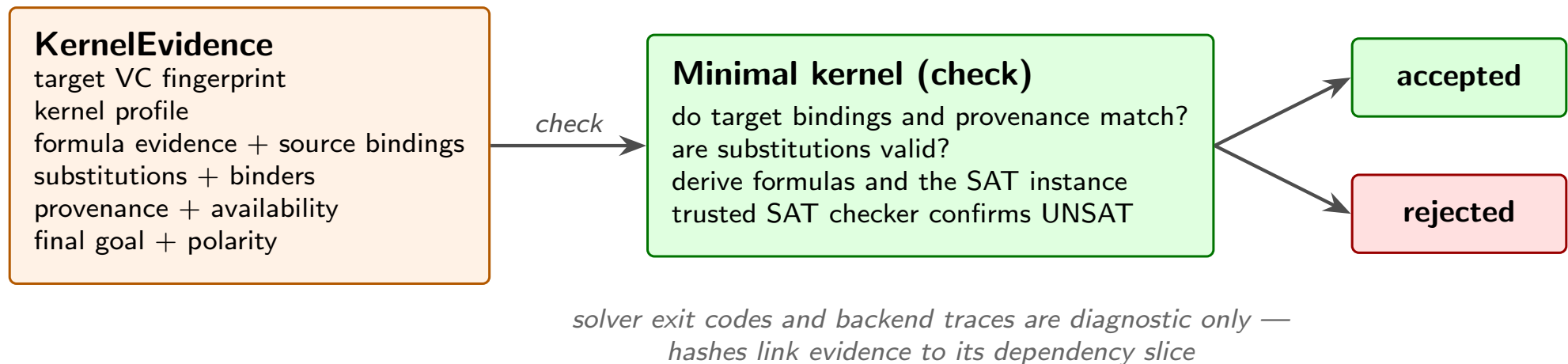
```
definition
  let T be type extends commutative associative unital Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

インスタンス化 **specification example** (必要な registration がある前提)

```
PermProduct[commutative associative unital AddMagma]
PermProduct[commutative associative unital MulMagma]
let R be commutative Ring; PermProduct[R qua AddMagma] :: additive view
```

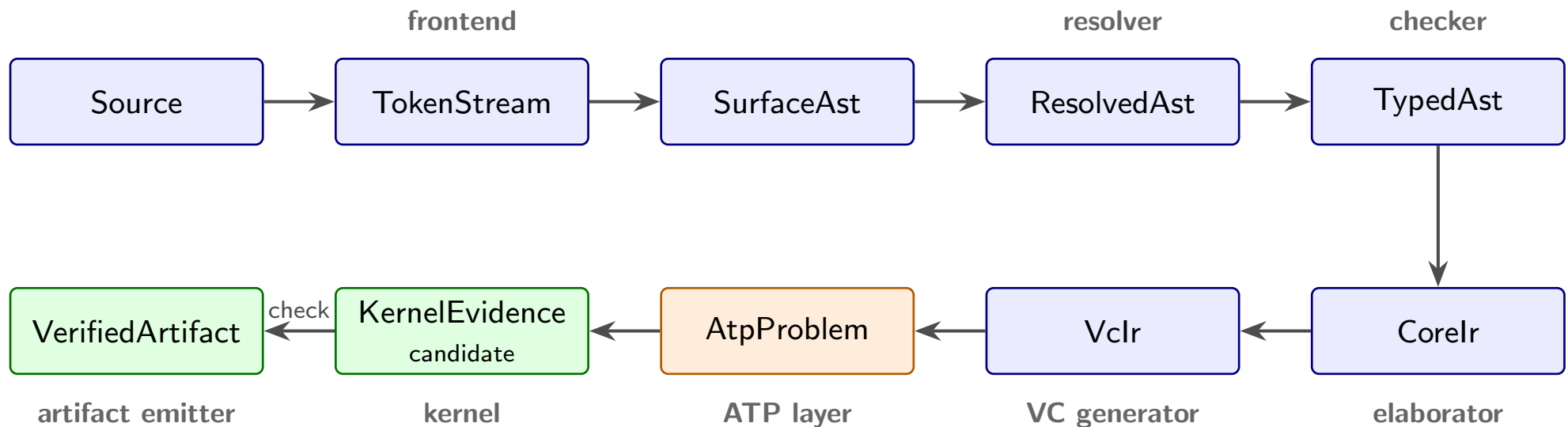
- 環から Magma への継承経路は二つ。qua によるビューの選択が、記法を決定する。

Backup 5. Kernel evidence と信頼できる SAT 検査



- evidence は元の論理式・明示的な代入・来歴・対象とゴールの対応を保持。kernel はこれらを検査し、決定的な SAT 問題を生成。信頼できるプロセス内 SAT 検査器で UNSAT を確認する。
- バックエンドの証明トレース、SMT の証明オブジェクト、ログ、終了コードは診断用のみ。

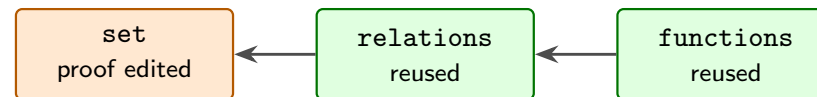
Backup 6. ATP の中心経路



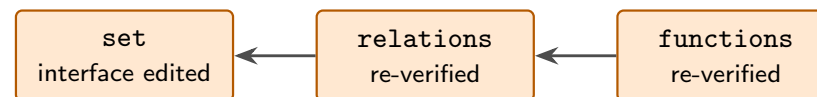
- 決定的な discharge 後の未解決義務のみを ATP に送る。前段の discharge にも再検査可能な evidence が必要。
- 各境界で、情報の管理主体・記録する成果物・変更後の再検査範囲を定める。

Backup 7. 差分検証

theorem proof edit:
public statement + accepted status unchanged — importers reuse



interface change:
the dependency cone re-verifies



reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance

- 証明本体の編集時、公開された主張と受理状態が不変なら、依存側の再ビルドは不要。インタフェース変更時は依存コーンを再検証。
- キャッシュ再利用には、関連キーの完全一致が必要。データ欠落時はキャッシュミス。再利用自体は証明の正当性を保証せず、クリーンビルドで全受理結果を再現する。

Backup 8. Package manifest

Mizar Evo **specification example**

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

- **再現可能なビルドの要件: 固定したソース・lockfile・ツールチェーン・検査器設定・決定的な ATP evidence。**
- **バージョン管理された再利用により、article 集合間の手作業コピーを解消。集約モジュールで、一分野を一つの import として公開できる。**

Source: doc/spec/en/23.package_management_and_build_system.md; Białystok 資料 Story 1。

Backup 9. 45 分版での追加

時間に応じ、以下の具体例を追加:

1. 単一のゴールで HOL から FOL への関数の符号化を説明 (app、部分適用、lambda lifting)。
2. Mizar の関数表現: Function of X, Y 、 $f.x$ 、集合論的対象としての soft type。
3. Template: 加法と乗法のビューを持つ PermProduct[T] (Backup 4)。
4. Algorithm: ユークリッドの互除法、by computation、関数子への昇格。
5. Structure とビュー: AddMagma、MulMagma、Magma (Białystok Story 2)。
6. 現代的な基盤: mizar.pkg、namespace、指紋グラフ (Backup 7-8)。
7. 検査: evidence のインスタンス化と SAT 検査 (本編 6.3、詳細は Backup 5)。

Backup 10. 出典と帰属

本講演における MML の原文引用:

目的	出典	行	使用箇所
関数適用 <code>f.x</code>	<code>funct_1.miz</code>	138-140	Frame 1.2
<code>soft type</code> としての Function of <code>X,Y</code>	<code>funct_2.miz</code>	87-90	Frame 1.2

Source URLs:

- `FUNCT_1`: https://mizar.uwb.edu.pl/version/current/mml/funct_1.miz
- `FUNCT_2`: https://mizar.uwb.edu.pl/version/current/mml/funct_2.miz

帰属についての注記:

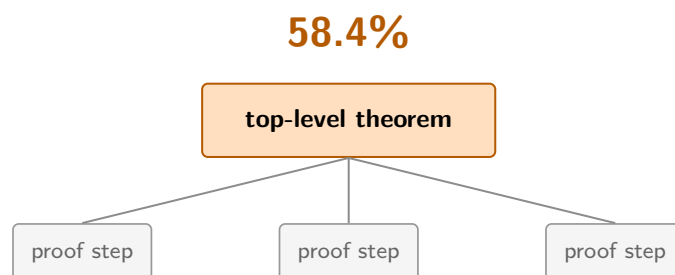
- MML のテキストは GPL-3.0-or-later / CC-BY-SA-3.0-or-later。article 名・URL・行番号を発表者ノートに記載。
- ベンチマークの数値は Backup 1-2 と `references.bib` を参照。最終版の作成前に、出版社の情報で書誌事項を確認。

Backup 11. 評価条件の比較

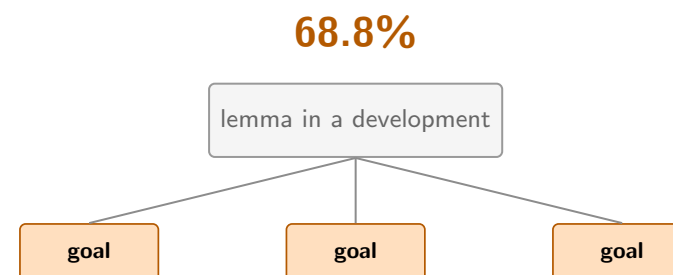
	MizAR 60 (ITP 2023)	Sledgehammer / AFP (ITP 2022)
報告された成功率	58.4% (1,690 / 2,896)	68.8% (3,440 / 5,000)
評価対象	MML 1147 の定理・補題。全 57,897 件のうち holdout 2,896 件	AFP の 50 entry から選んだ局所ゴール 5,000 件
前提選択	ライブラリ全体から学習的に選択	MePo。基準 512 事実、構成により増減
時間予算	ポートフォリオ合計 420 CPU 秒	greedy 16 構成 × 各 30 CPU 秒 = 480 CPU 秒
成功の判定	ATP 証明の発見 (hammering mode)	外部 prover の証明発見。Isabelle 内の再構成は評価対象外

- 評価対象・前提選択・時間予算・構成選択が異なる。成功率だけでは設計上の優劣を判定できない。

Backup 12. 評価単位: 定理全体と局所ゴール



Mizar 60 (ITP 2023) counts *whole*
top-level theorems and lemmas
2,896 holdout problems from 57,897 in
MML 1147; learned premises; 420 CPU s



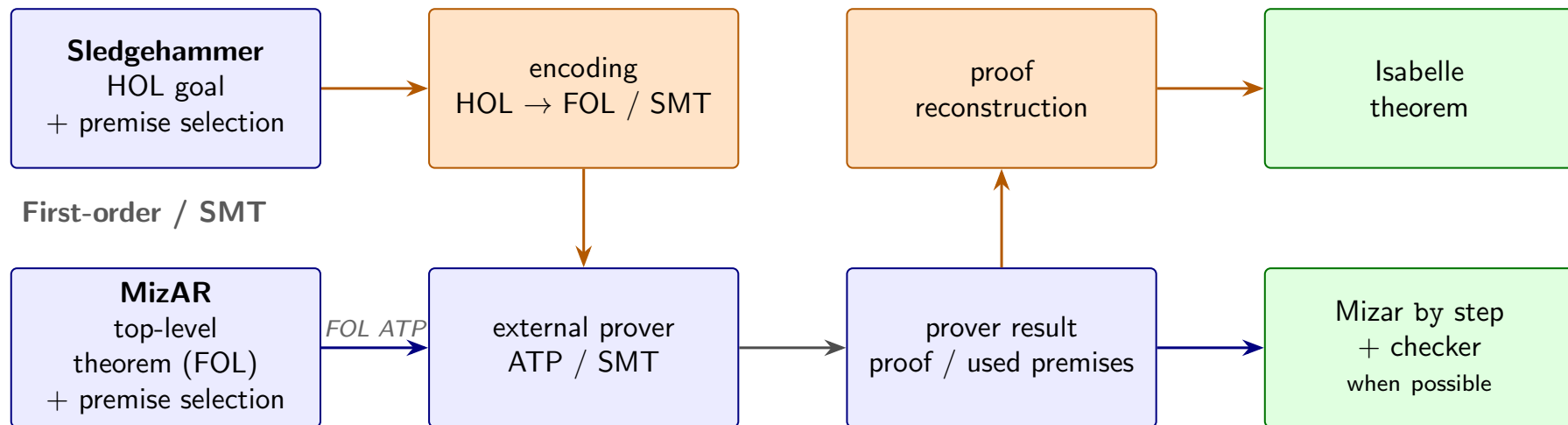
Sledgehammer / AFP (ITP 2022)
counts *proof goals as they arise*
5,000 goals from 50 entries; greedy 16 con-
figurations × 30 CPU s; no reconstruction

Different units, premise selection, and budgets.

- MizAR: 著者による前提指定なしに、ライブラリから定理全体を自動証明できるか。
- AFP 研究: 証明中の局所ゴールを、その文脈で自動証明できるか。多くは人手で記述された証明の内部に位置する。
- いずれも妥当な評価であるが、対象とする課題は異なる。

Backup 13. 一階 ATP への二つの接続経路

Higher-order (Isabelle/HOL)



Mizar Evo: kernel evidence checking replaces the last step (later slide).

- Sledgehammer の一階・SMT 経路: 高階のゴールを翻訳し、得られた証明を Isabelle 内で再構成する。
- MizAR: 一階・集合論の問題を ATP へ送る。

Backup 14. 高階論理における関数

HOL の文の概形 **sketch** (Isabelle/HOL 風の記法)

```
f :: 'a => 'b           x :: 'a
```

```
P (f x)
```

- HOL: 関数型と関数適用を論理に組み込む。
- 関数を引数・戻り値とする関数、部分適用、ラムダ抽象を直接記述できる。
- 高階の構造を用いた簡潔な数学的記述が可能。
- E や Vampire の一階推論を利用するには、高階構造を一階表現へ符号化する必要がある。

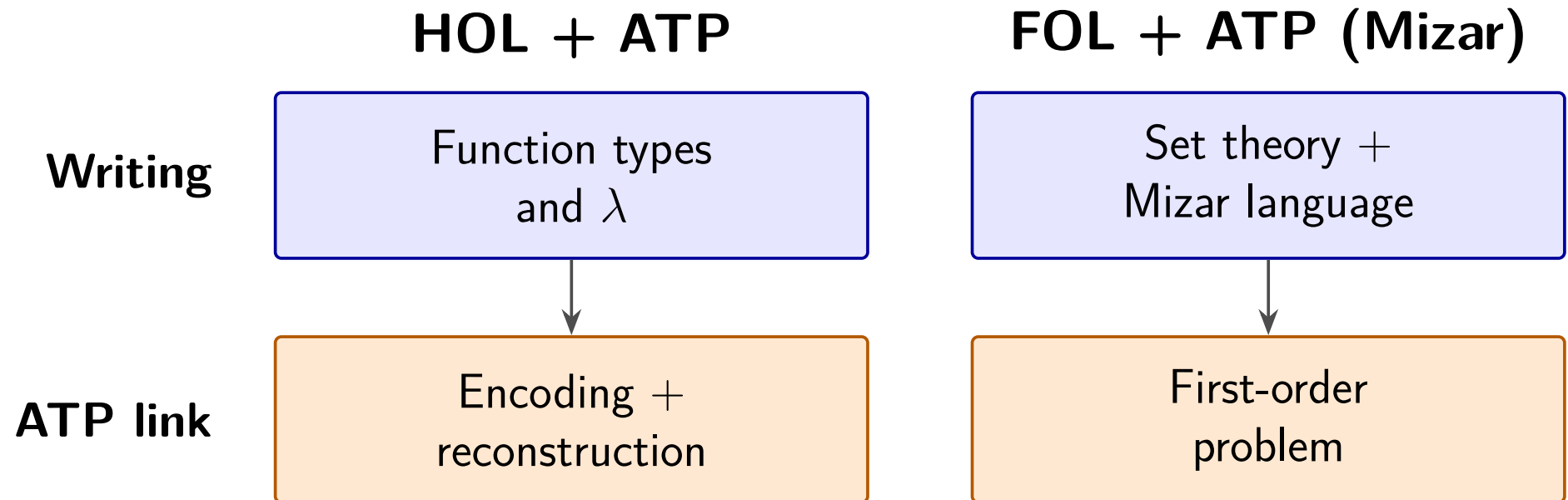
Backup 15. HOL の符号化と再構成

符号化の手順の概形 sketch

$F\ X$	$\rightarrow$	$\text{app}(F, X)$	
$(\%x. t) \dots$	$\rightarrow$	fresh constant + defining axioms	(lambda lifting)
polymorphic types	$\rightarrow$	type guards or type tags	
Boolean-valued terms	$\rightarrow$	extra encoding	

- 関数変数の適用は `app` で明示。ラムダ抽象は `lambda lifting` またはコンビネータで、型は `guard` または `tag` で符号化する。
- ATP の証明を Isabelle 内で再構成 (reconstruction) : `metis`、`smt` の `replay`、生成した `Isar` テキストを利用。
- 高階 ITP と一階 ATP の接続を実現する技術。
- 接続の仕組みを示す説明であり、実測上の性能の優劣を示すものではない。

Backup 16. 複雑さを担う層の違い



- HOL は高階構造を ATP 向けに符号化。Mizar は集合論の細部を言語層で抽象化。
- Mizar Evo: 一階の基盤を保ち、数学的な記述を現代化。

Backup 17. 設計上のトレードオフ

	HOL ITP + ATP	FOL ITP + ATP
記述	高階機能による簡潔な記述	直接記述すると冗長
関数	基本的な高階の対象	一階の集合論的对象
ATP 接続	高階構造の一階・SMT 符号化	一階問題の生成
再構成	Isabelle 内での再構成	対応可能な Mizar ステップの再検査
言語層の役割	HOL の抽象	一階の細部の抽象化

- 表現と検査の経路を比較した表であり、実測上の性能優位性を示すものではない。